DOCUMENTAÇÃO DO PROJETO

ENGENHARIA DE SOFTWARE

Projeto Integrador V – Aplicativos Gamificados

NEXUS-RPG

Nome da Empresa (Grupo): *Nexus RPG*
Curso: *Engenharia De Software*
Período: *Matutino*
Instituição Parceira: *[Preencher aqui]*
Professor Orientador: *Brendo Vale*
Semestre Letivo: *5°Semestre*

EQUIPE:

#	Nome Completo	Função no Projeto	Modalidade
1	Guilherme Augusto	PO / Dev Frontend	Presencial
2	Flávio Gimenes	Dev Frontend / UX/UI Designer	Presencial
3	Dhiogo Nascimento	Dev Full-Stack	Presencial
4	Matheus Okada	Dev Backend	Presencial
5	~~~~	~~~~	~~~~

HISTÓRICO DE VERSÕES

Registrem todas as alterações significativas no documento. Isso demonstra a evolução do projeto e é uma boa prática de engenharia.

Data	Versão	Descrição da Alteração	Autor
06/03/2026	1.0	*Criação do documento e definição do problema*	*Flávio Gimenes*
13/03/2026	2.0	*Engenharia e Design, Elaboração da solução e Arquitetura técnica*	*Flávio Gimenes*
23/03/2026	3.0	Construção, qualidade e implementação do código	*Flávio Gimenes*

FASE 1: DESCOBERTA E DEFINIÇÃO (INCEPTION)

1.1 Contextualização e Problema

Descreva o problema que o aplicativo vai resolver. Seja específico: quem sofre com esse problema? Onde? Com que frequência? Qual o impacto?

Problema: *Atualmente, existe uma barreira de entrada significativa no aprendizado de programação. Métodos tradicionais, baseados em vídeos longos e documentações densas, falham em engajar jovens e iniciantes que estão acostumados com estímulos rápidos e interativos.*

Quem sofre com o problema? *Estudantes iniciantes, entusiastas de tecnologia e pessoas em transição de carreira que possuem interesse na área, mas sofrem com a falta de motivação e tédio nos métodos de ensino convencionais.*

Onde e com que frequência? *O problema ocorre diariamente em ambientes de estudo autônomo e em cursos técnicos/superiores, onde a taxa de evasão nos primeiros meses é altíssima devido à complexidade percebida e à falta de dinamismo.*

Impacto: *Isso gera uma escassez de profissionais qualificados no mercado de tecnologia e a frustração de indivíduos que desistem de carreiras promissoras por acreditarem que "programação não é para eles".*

1.2 Público-Alvo e Personas

Defina quem são os usuários do sistema. Criar personas ajuda a equipe a tomar decisões de design centradas no usuário.

Persona	Perfil	Dor Principal	Necessidade
Lucas, 18 anos	Estudante de ensino médio e gamer nas horas vagas.	Acha aulas de lógica de programação chatas e teóricas demais.	Praticar conceitos de código enquanto se diverte e progride em um jogo.
Beatriz, 24 anos	Jovem profissional querendo transição de carreira para Tech.	Chega cansada do trabalho e não tem energia para ler manuais técnicos.	Um método de aprendizado leve, que pareça um hobby/descanso e não "mais trabalho".
Enzo, 15 anos	Adolescente curioso sobre como criar seus próprios games.	Sente-se intimidado pela complexidade da sintaxe das linguagens.	Aprender a lógica de forma visual e gamificada, recebendo recompensas por erros e acertos.

1.3 Alinhamento com os ODS (ONU)

Todo projeto de extensão precisa ter impacto social. Conecte sua solução a um Objetivo de Desenvolvimento Sustentável da ONU.

ODS Principal: *ODS 4 – Educação de Qualidade.*

Justificativa (como a solução técnica impacta as metas deste ODS?): O Nexus RPG impacta diretamente a meta de aumentar o número de jovens e adultos com habilidades relevantes, inclusive competências técnicas e profissionais, para o emprego e empreendedorismo. Ao gamificar o ensino de programação, o projeto democratiza o acesso ao conhecimento tecnológico, tornando-o inclusivo e atrativo para perfis que não se adaptam ao ensino formal, combatendo a evasão escolar na área de exatas e preparando mão de obra para a economia digital.

1.4 Validação e Vídeo de Pitch

O pitch é a apresentação da ideia em formato de vídeo curto. Ele deve convencer o espectador de que o problema é real e a solução é viável.

Link do Vídeo (YouTube): https://youtu.be/pB7OxoXJBXk?si=XrrwF8tRUaF31VJ8

1.5 Checklist de Entrega da Fase 1

[X] - Problema claramente definido e contextualizado
[X] - Pelo menos 2 personas criadas
[X] - ODS identificado com justificativa
[X] - Vídeo de pitch gravado e publicado

FASE 2: ENGENHARIA E DESIGN (ELABORAÇÃO)

2.1 Metodologia de Desenvolvimento

Escolham qual metodologia ágil será usada e expliquem como vão organizar o trabalho da equipe.

Metodologia escolhida (Scrum, Kanban, XP, etc.): Scrum (com adaptações de Kanban para o fluxo de tarefas)

Duração dos sprints: *2 Semanas*

Ferramentas de gestão (Trello, Jira, GitHub Projects, etc.): GitHub Projects (para o board de tarefas) e **Discord** (para comunicação e reuniões).

Fluxo de trabalho: A equipe realizará uma **Sprint Planning** no início de cada ciclo para definir as tarefas. Teremos **Dailies** assíncronas via Discord para reportar progresso. Ao final de cada sprint, faremos uma **Review** para testar as mecânicas de jogo implementadas e uma **Retrospective** para ajustar o processo de desenvolvimento.

2.2 Requisitos do Sistema

2.2.1 Requisitos Funcionais (RF)

Definem O QUE o sistema faz. São as funcionalidades visíveis para o usuário. Use critérios SMART: Específico, Mensurável, Atingível, Relevante e com prazo.

ID	Descrição (SMART)	Prioridade	Ator Principal	Sprint
RF01	O sistema deve permitir login via OAuth2 em menos de 10s	*Essencial*	*Usuário*	1
RF02	O sistema deve apresentar diálogos com o NPC "Viajante" para entrega de quests.	*Essencial*	*Usuário*	1
RF03	O sistema deve carregar uma batalha com Boss onde perguntas de código são exibidas.	*Essencial*	*Usuário*	2
RF04	O sistema deve subtrair HP do Boss em 20% para cada resposta correta.	*Essencial*	*Sistema*	2
RF05	O sistema deve subtrair HP do Usuário para cada resposta errada.	*Essencial*	*Sistema*	2
RF06	O sistema deve exibir um ranking global de "XP" e níveis dos jogadores.	Importante	*Usuário*	3

2.2.2 Requisitos Não Funcionais (RNF)

Definem COMO o sistema deve funcionar. São qualidades invisíveis para o usuário final, mas fundamentais: desempenho, segurança, usabilidade, acessibilidade.

ID	Descrição (SMART)	Categoria	Métrica de Aceite
RNF01	Suportar 100 acessos simultâneos sem queda	*Escalabilidade*	Tempo resposta < 2s
RNF02	Toda comunicação com o BD deve ser criptografada	*Segurança*	TLS 1.3 ativo
RNF03	A interface deve ser responsiva para Web e Mobile.	*Usabilidade*	Compatibilidade 100%
RNF04	O tempo de carregamento de uma batalha não deve exceder 3s.	*Desempenho*	Teste de carga

2.2.3 Requisitos de Domínio (RD)

São regras que vêm do mundo real: leis, regimentos, normas do MEC, LGPD. O sistema precisa obedecer essas regras, mesmo que o cliente não peça explicitamente.

ID	Descrição	Origem/Regra	Impacto no Sistema
RD01	Cálculo de média segue o regimento da escola	*Regimento Escolar*	Módulo de notas
RD02	Dados de menores exigem consentimento dos pais	*LGPD / ECA*	Fluxo de cadastro
RD03	Proteção de dados sensíveis dos menores de idade.	*LGPD*	Módulo de Cadastro
RD04	Conteúdo pedagógico alinhado com trilhas de lógica.	*MEC / SBC*	Banco de Questões

2.2.4 Histórias de Usuário (User Stories)

User Stories traduzem requisitos na perspectiva do usuário. Formato: 'Como [quem], eu quero [o quê], para [por quê].'

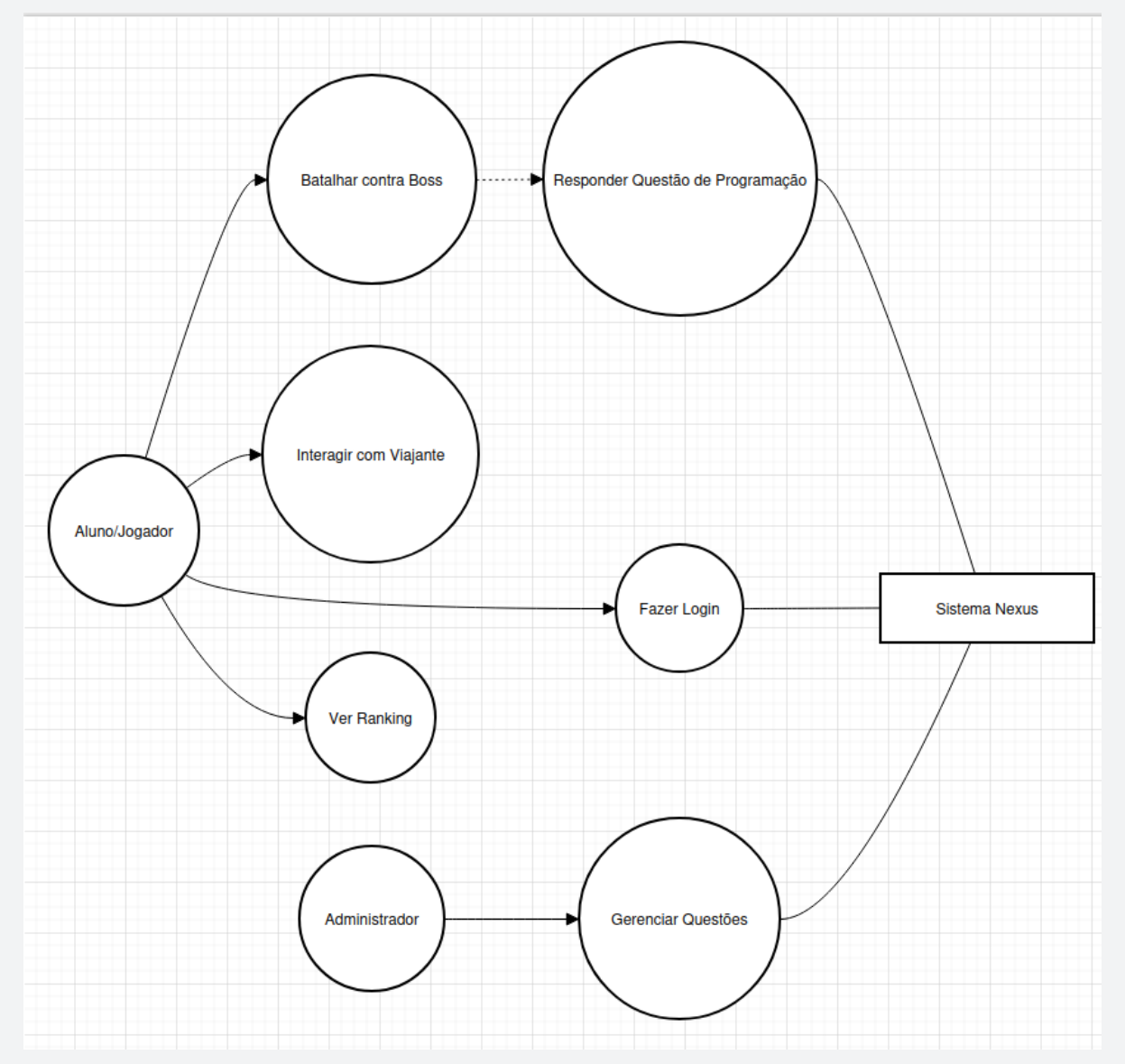
#	Como... (Quem)	Eu quero... (O Quê)	Para... (Por Quê)
US01	*Aluno*	*Ver meus pontos e ranking*	*Me motivar a estudar mais*
US02	*Professor*	*Cadastrar atividades*	*Acompanhar o progresso da turma*
US03	*Aluno*	*"Lutar" contra um Boss de Python*	*Validar meus conhecimentos de forma divertida*
US04	*Aluno*	*Receber dicas do Viajante ao errar*	*Aprender com o erro e tentar novamente*

2.3 Modelagem de Software (UML)

Os diagramas UML são a planta baixa do software. Assim como um engenheiro civil não constrói sem planta, um engenheiro de software não codifica sem modelagem.

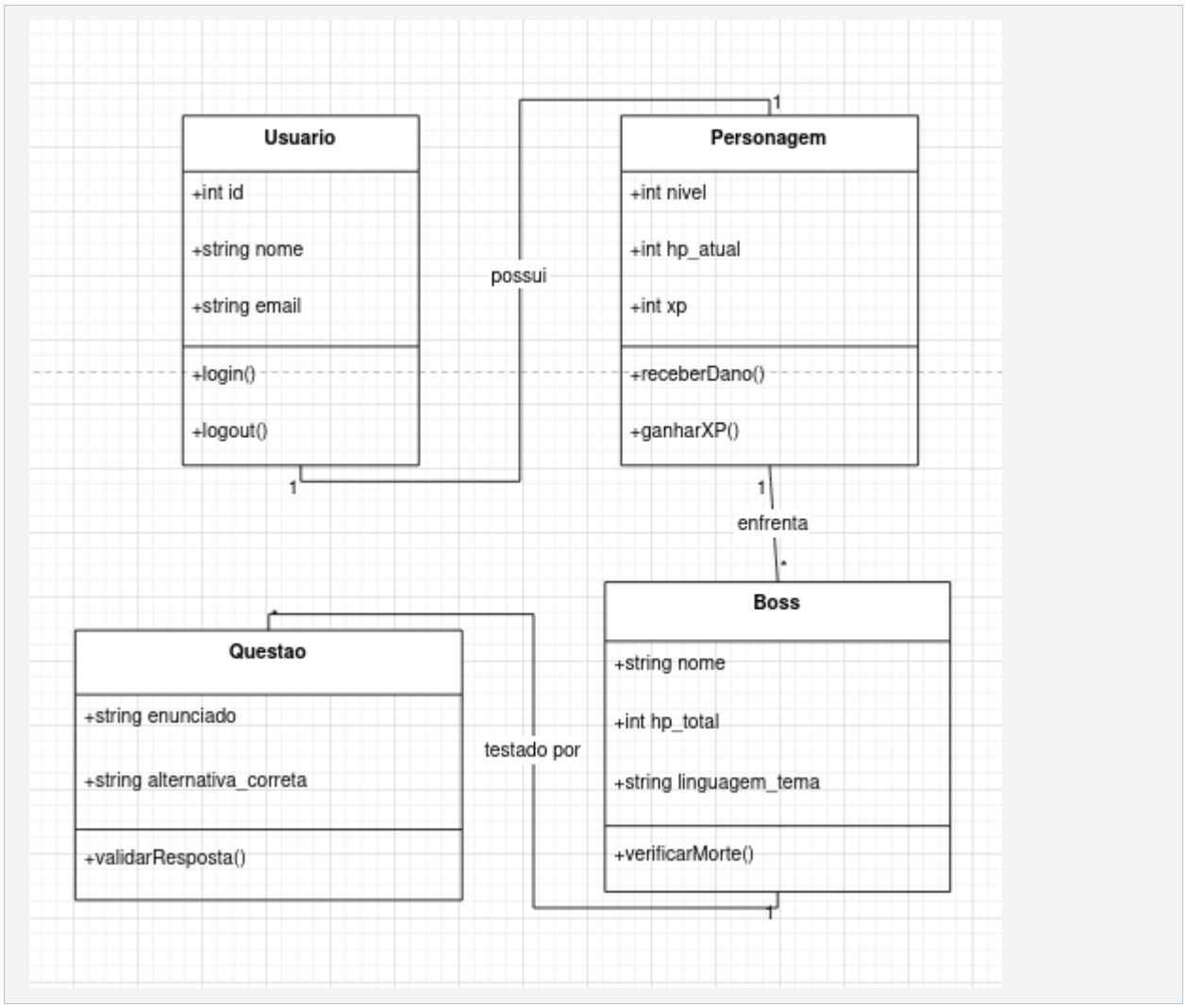
2.3.1 Diagrama de Casos de Uso

Mostra QUEM usa o sistema e O QUÊ cada ator pode fazer. É o mapa de funcionalidades.



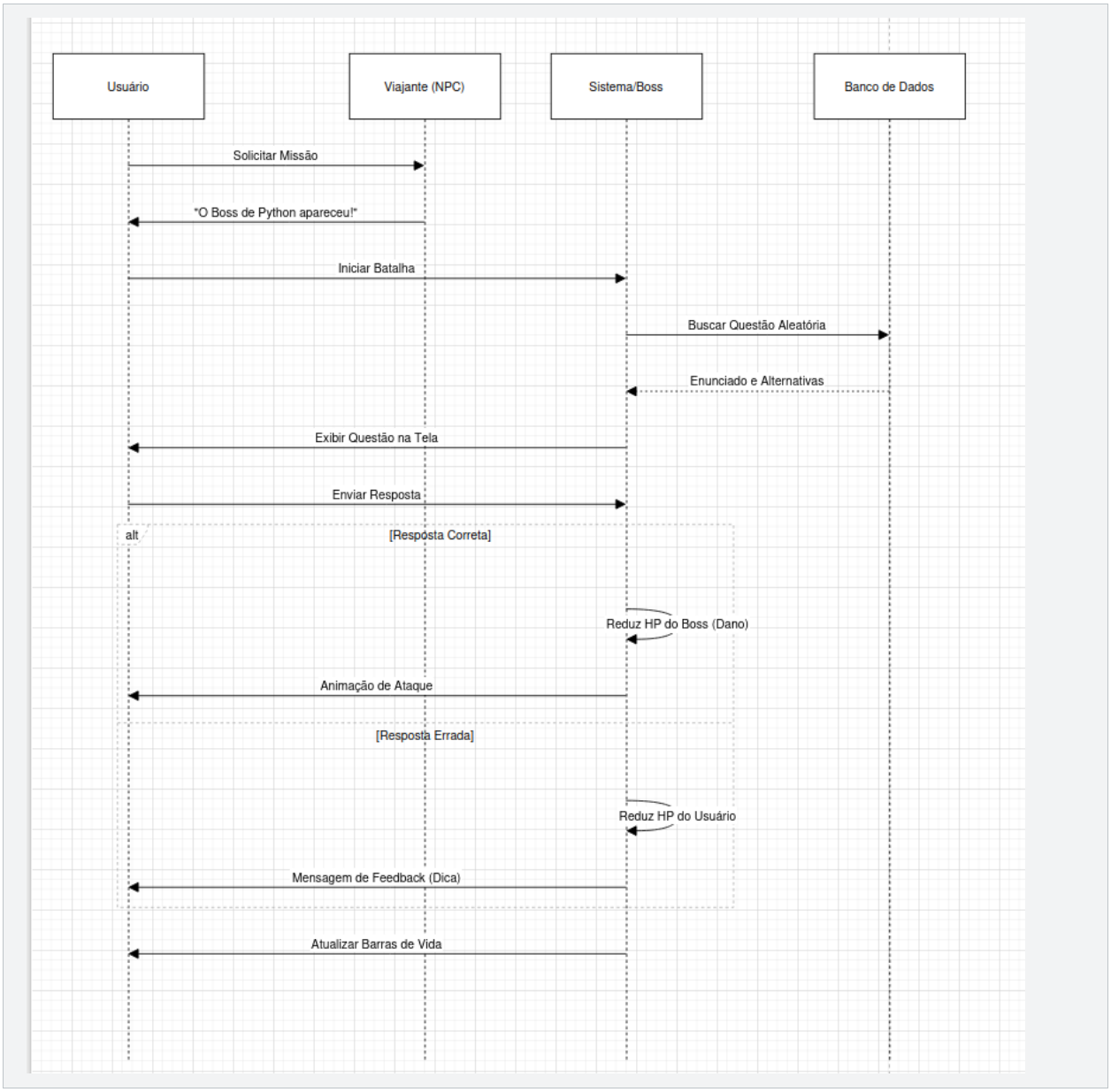
2.3.2 Diagrama de Classes / Entidade-Relacionamento

Mostra a estrutura de dados: quais entidades existem, seus atributos e como se relacionam.



2.3.3 Diagrama de Sequência (Opcional, mas recomendado)

Mostra a ordem das mensagens entre objetos para um cenário específico. Útil para fluxos complexos como autenticação ou transações.



2.4 Arquitetura e Padrões

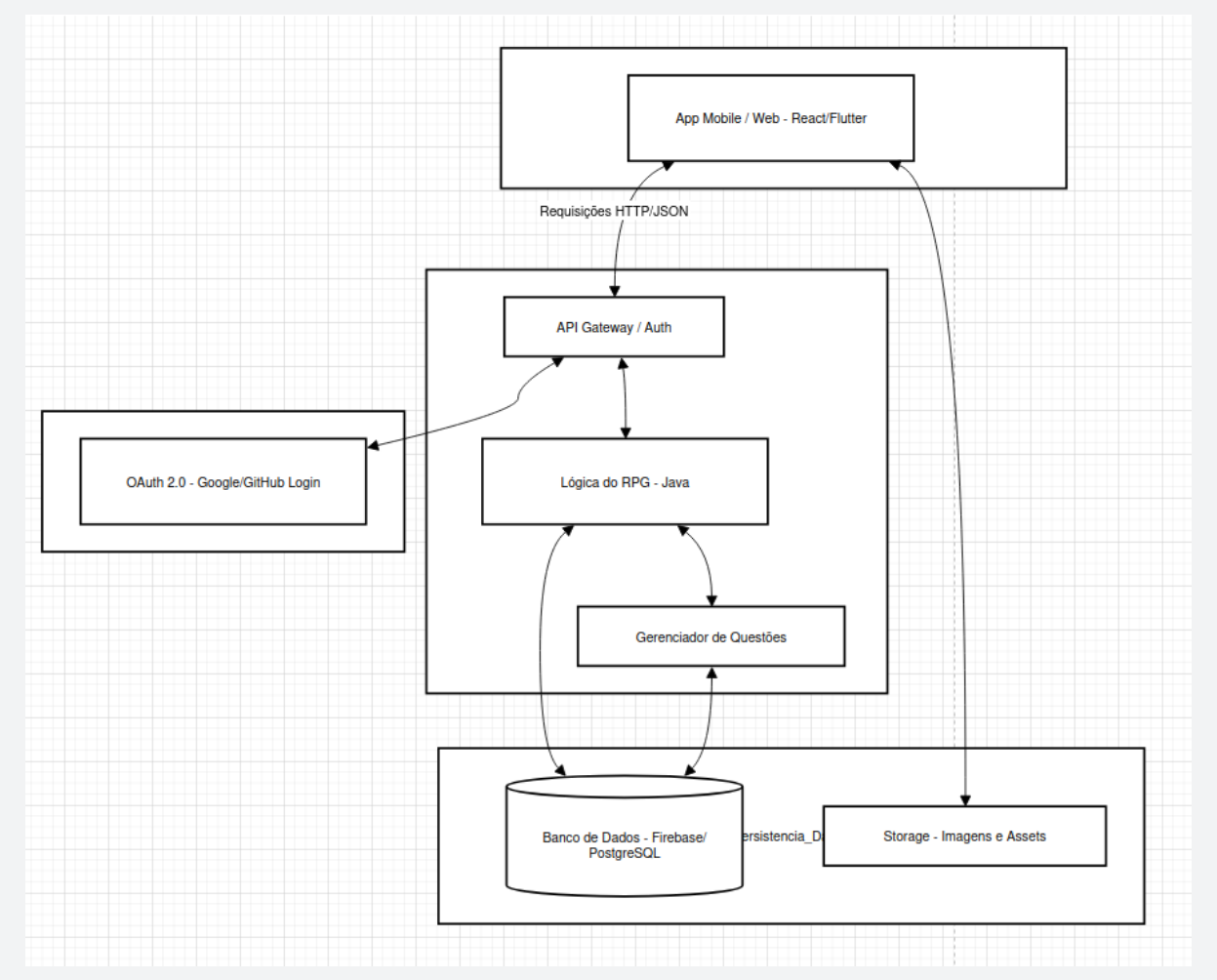
A arquitetura define como as partes do sistema se organizam. É a decisão mais importante do projeto, pois é difícil mudar depois.

Padrão Arquitetural (Ex: MVVM, Clean Architecture, MVC, Layered): MVC (Model-View-Controller).

Justificativa: Como o projeto separa claramente os dados (questões/personagens), a interface do jogo (view) e a lógica de combate (controller), o MVC facilita a manutenção e permite que a parte visual do RPG seja atualizada sem quebrar a lógica de ensino.

2.4.1 Diagrama de Arquitetura

Represente visualmente as camadas ou módulos do sistema e como eles se comunicam.



2.5 Gestão de Riscos

Todo projeto tem riscos. O engenheiro de software se diferencia por antecipá-los e ter planos de contingência. Identifiquem pelo menos 4 riscos.

#	Risco	Probabilidade	Impacto	Plano de Mitigação
R01	*Baixa adesão dos usuários*	*Média*	*Alto*	*Gamificação robusta e UX simplificada*
R02	*Atraso no desenvolvimento*	*Alta*	*Médio*	*Uso de componentes prontos e sprints curtos*
R03	Perguntas se tornarem repetitivas	*Média*	*Médio*	*Banco de questões randômico e atualizável*

#	Risco	Probabilidade	Impacto	Plano de Mitigação
R04	Incompatibilidade em navegadores	*Baixa*	*Médio*	*Testes automatizados em diferentes browsers*

2.6 Planejamento Estratégico e Viabilidade

2.6.1 Modelo de Negócio

Mesmo sendo um projeto acadêmico, pensar no modelo de negócio é exercitar o pensamento de engenheiro empreendedor.

Modelo escolhido (SaaS, B2B, Freemium, Licenciamento, Open Source, etc.): *Freemium*

Justificativa: O conteúdo base de lógica é gratuito para democratizar o ensino (alinhado ao ODS 4), mas itens cosméticos para o personagem ou trilhas avançadas de linguagens específicas podem ser pagos.

2.6.2 Canvas Simplificado do Projeto

Proposta de Valor	Segmento de Clientes	Canais
Transformar o aprendizado de programação em uma experiência imersiva e viciante, reduzindo a frustração inicial e aumentando a retenção do conhecimento através de mecânicas de RPG.	Estudantes de 15 a 25 anos; entusiastas de jogos que desejam aprender uma nova habilidade; escolas técnicas que buscam ferramentas complementares de ensino.	Lojas de aplicativos (Google Play/App Store); redes sociais (TikTok/Instagram); comunidades de desenvolvedores (Discord/Reddit).
Fontes de Receita	**Estrutura de Custos**	**Métricas-Chave**
Modelo *Freemium* (conteúdo base gratuito); venda de itens cosméticos (skins para o herói); passes de batalha com trilhas de linguagens avançadas (ex: Java, Rust).	Hospedagem de servidores e banco de dados; licenças de softwares de design; marketing digital; manutenção e atualização de conteúdo (novas quests).	Taxa de retenção (usuários que voltam no dia seguinte); Tempo médio de sessão; Taxa de acerto nas batalhas contra bosses; Número de novos cadastros por semana.

2.6.3 Investimento Inicial (CapEx)

Recurso	Descrição	Valor (R$)
Engenharia	2 Desenvolvedores Full-stack (Nível Júnior/Pleno) por 3 meses.	R$ 56.000,00
Design	Criação de UI/UX, Sprites de personagens e bosses	R$ 3.500,00
Software/Assets	Softwares de Design e Infra de Testes.	R$ 3.000,00
Equipamentos	Notebooks e Computadores	R$ 20.000,00
TOTAL		**R$ 82.500,00**

2.6.4 Custo Operacional Mensal (OpEx)

Serviço	Finalidade	Valor Mensal (R$)
Servidores	Instâncias de Servidor, Banco de Dados Relacional e CDN.	R$ 450,00
Manutenção	Reserva para correções e suporte técnico (estimado em horas).	R$ 800,00
Marketing	Criação de identidade visual (Branding) e tráfego pago inicial.	R$ 5.000,00
Suporte	2 Estagiário/Monitor (Part-time) para dúvidas e tickets.	R$ 2.400,00
TOTAL		**R$ 8.650,00**

2.6.5 Cronograma de Entregas

Fase	Entrega Esperada	Atividades Principais	Prazo Final
Fase 1	*Validação do Problema*	*Pesquisa, Personas, Pitch*	*10/03/26*
Fase 2	*Design e Planejamento*	*Requisitos, UML, Arquitetura*	*17/03/26*
Fase 3	*Versão Alpha*	*Desenvolvimento e CI/CD*	*24/03/26*
Fase 4	*Lançamento e Testes*	*Testes com usuários, ajustes finais*	*31/03/26*

Checklist de Entrega da Fase 2

[X] - Metodologia definida com ferramentas e ritos
[X] - Pelo menos 5 Requisitos Funcionais documentados
[X] - Requisitos Não Funcionais e de Domínio preenchidos
[X] - User Stories escritas para os RFs principais
[X] - Diagramas UML criados (Casos de Uso + Classes/ER)
[X] - Arquitetura definida e justificada
[X] - Pelo menos 4 riscos identificados com planos de mitigação
[X] - Cronograma e viabilidade financeira preenchidos

FASE 3: CONSTRUÇÃO E QUALIDADE (IMPLEMENTAÇÃO)

3.1 Stack Tecnológica

Definam todas as tecnologias usadas no projeto e justifiquem cada escolha.

Camada	Tecnologia	Justificativa
Frontend Mobile	React Native / Expo	*Desenvolvimento cross-platform (Android/iOS) com alta performance e grande comunidade.*
Backend	Typescript / Java	*Arquitetura que permite escala automática e rapidez no desenvolvimento de APIs.*
Banco de Dados	Firestore / PostgreSQL	*Firestore para dados em tempo real (combates) e PostgreSQL para relações complexas (questões).*
Autenticação	Firebase Auth / OAuth2	*Segurança de nível industrial e facilidade para login com Google/GitHub.*
Versionamento	GitHub	*Padrão de mercado para controle de versões e colaboração entre a equipe.*
Design de UI/UX	Figma	*Prototipagem de alta fidelidade para validar a interface gamificada antes de codar.*

3.2 Pipeline de CI/CD

CI/CD (Integração Contínua / Entrega Contínua) automatiza testes e deploy. Descreva como vocês vão do código ao app publicado.

O fluxo de trabalho do **Nexus RPG** utiliza a integração do **GitHub** com o ecossistema **Expo** para garantir entregas rápidas e código estável.

- **Hospedagem e Controle de Versão:** O código-fonte está hospedado no **GitHub**. Utilizamos o fluxo de **GitHub Flow**, onde a branch main contém o código estável e as branch/”branchFrontEnd" (por exemplo) são usadas para novos desenvolvimentos. Todo código só entra na main após um *Pull Request* revisado.
- **CI (Integração Contínua):** Configuramos um **GitHub Action** que é disparado a cada *Push* ou *Pull Request*. Ele realiza automaticamente:
    - A instalação das dependências (npm install).

  - A verificação de erros de sintaxe e padrões de código (**Linter**).
  - A execução de testes unitários básicos (se houver).
- **CD (Entrega Contínua):** Para facilitar os testes sem a necessidade de gerar um APK/IPA a cada mudança, utilizamos o **EAS Update**.
  - Sempre que um novo recurso é finalizado, o desenvolvedor publica uma atualização no canal de desenvolvimento do Expo.
  - **Deploy para Testes:** A equipe e o professor podem visualizar as mudanças em tempo real abrindo o **Expo Go** em seus dispositivos móveis e acessando o link do projeto, garantindo um feedback imediato sem burocracia de instalação.

3.3 Repositório e Documentação

Link do GitHub: https://github.com/dhiogonasc/Nexus-RPG

3.3.1 Manual de Instalação

Instruções para que qualquer desenvolvedor consiga rodar o projeto localmente.

Para rodar o projeto localmente e contribuir com o reino de Nexus, siga os passos abaixo:

1. **Pré-requisitos:** Certifique-se de ter o **Node.js (LTS)**, **Git** e o **Expo Go** (no celular) instalados.
2. **Clonagem:** No terminal, execute: git clone https://github.com/dhiogonasc/Nexus-RPG.git
3. **Dependências:** Entre na pasta do projeto e instale os pacotes com: npm install.
4. **Variáveis de Ambiente:** Crie um arquivo .env na raiz seguindo o modelo .env.example, inserindo suas chaves do Firebase.
5. **Execução:** Inicie o servidor de desenvolvimento com: npx expo start. Escaneie o QR Code com o app Expo Go para testar no seu dispositivo físico.

3.4 Padrões de Código e Boas Práticas

Defina as convenções que a equipe vai seguir para manter o código consistente e legível.

Aspecto	Convenção Adotada
Nomenclatura	*camelCase para variáveis, PascalCase para componentes*

Aspecto	Convenção Adotada
Commits	Conventional Commits (ex: feat:, fix:, docs:, style:).
Code Review	Pelo menos 1 aprovação de outro colega antes do merge na branch main.
Linting/Style	Uso de **ESLint** e **Prettier** (Ferramentas muito boas no desenvolvimento JavaScript) para manter a padronização visual e evitar erros bobos.
Documentação	Comentários em **JSDoc** para explicar a lógica de funções complexas (ex: cálculo de dano).

Checklist de Entrega da Fase 3

[X] - Stack tecnológica definida e justificada
[X] - Repositório GitHub com commits regulares da equipe
[X] - Pipeline de CI/CD configurado (mesmo que básico)
[X] - Manual de instalação funcional
[X] - Versão Alpha do app funcional

FASE 4: VALIDAÇÃO E ENTREGA (TRANSIÇÃO)

INFO O QUE É ESTA FASE?: O software está pronto? Só os usuários reais podem confirmar. Nesta fase, vocês testam com pessoas de verdade, medem o impacto e documentam os resultados.

4.1 Estratégia de Testes

Teste é garantia de qualidade. Defina quais tipos de testes serão aplicados e como.

4.1.1 Testes de Software

Tipo de Teste	O Que Testa	Ferramenta/Abordagem
Unitário	*Funções e componentes isolados*	*Jest, React Testing Library*
Integração	*Comunicação entre módulos*	*Supertest, Firebase Emulators*
Manual/E2E	*Fluxos completos do usuário*	*Roteiro de testes manuais*
[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]

4.1.2 Testes de Usabilidade

Sessões com usuários reais da escola parceira para validar se o app é fácil de usar e resolve o problema.

#	Perfil do Testador	Tarefa Testada	Resultado	Observações
1	*Aluno, 15 anos*	*Fazer login e ver notas*	*Sucesso em 30s*	*Botão muito pequeno*
[Preencher]	[Preencher]	[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]	[Preencher]	[Preencher]

4.2 Indicadores de Impacto

Métricas concretas que provam que o projeto resolveu (ou contribuiu para resolver) o problema definido na Fase 1.

Indicador	Meta	Resultado Real	Evidência
Adesão dos alunos	*> 70% da turma*	*[Preencher após testes]*	*Screenshot / Analytics*

Indicador	Meta	Resultado Real	Evidência
Satisfação (NPS)	*> 8/10*	*[Preencher após testes]*	*Formulário Google*
[Preencher]	[Preencher]	[Preencher]	[Preencher]
[Preencher]	[Preencher]	[Preencher]	[Preencher]

4.3 Lições Aprendidas

Reflitam sobre o processo: o que deu certo, o que fariam diferente, e o que aprenderam como engenheiros de software.

Descrevam as principais lições: desafios técnicos superados, dinâmica da equipe, relação com a escola parceira, habilidades desenvolvidas...

4.4 Conclusão

Escrevam uma conclusão conectando o problema da Fase 1 com os resultados da Fase 4. Como a engenharia de software ajudou a resolver o problema? O que o projeto trouxe de valor para a comunidade?

Checklist de Entrega da Fase 4

[ ] - Testes de software executados e documentados
[ ] - Testes de usabilidade realizados com usuários reais
[ ] - Indicadores de impacto preenchidos com dados reais
[ ] - Lições aprendidas documentadas
[ ] - Conclusão final escrita
[ ] - App funcional e demonstrável

REFERÊNCIAS

ATENÇÃO: Todo projeto com indícios de plágio será REPROVADO. Citem todas as documentações, tutoriais, artigos e bibliografias utilizados. Use o formato ABNT.

MEIRA, Luciano. Ludicidade, jogos digitais e gamificação na aprendizagem. Porto Alegre: Grupo A, 2019.

Adicionem aqui as referências do projeto. Incluam: documentações oficiais (React Native, Firebase, etc.), artigos, livros, tutoriais e qualquer material que tenha sido utilizado.

NOTA DO PROFESSOR

INFO ORIENTAÇÃO FINAL: Este template foi projetado para guiar vocês na criação de um projeto real de engenharia de software. Não se trata apenas de codificar um app, mas de projetar um sistema sustentável. Sigam cada fase com atenção, preencham com honestidade e lembrem-se: no mercado, vocês lidarão com documentos como SAD (Arquitetura), PRD (Requisitos) e Planos de Teste complexos. Este documento condensa o essencial com o rigor técnico esperado de um engenheiro de software.

Professor Orientador: Brendo Vale

Semestre: 2026/1